



CAHIER DES CHARGES

Backend hybride

API centrale · Sécurité · Passerelle IA



PROJET	KWISMO — Protection anti-fraude Mobile Money
DOCUMENT	Cahier des charges — Backend
VERSION	1.0
DATE	21 juillet 2026
DESTINATAIRE	Développeur Backend & Intégration
STACK CIBLE	Python · FastAPI · Prisma · Swagger · Docker

Table des matières

1	Présentation du document.....	4
1.1	Rôle du backend dans l'architecture	4
1.2	Objectifs de qualité	4
2	Stack technologique.....	5
3	Architecture & structure des dossiers	6
3.1	Choix : architecture en couches (Layered / Clean léger).....	6
3.2	Arborescence normative.....	6
3.3	Rôle des dossiers clés — sécurité, données, il8n.....	7
4	Modèle de données (Prisma)	8
5	Modules fonctionnels & routes API	9
5.1	Authentification (auth)	9
5.2	Utilisateurs (users).....	9
5.3	Numéros (numbers).....	9
5.4	Signalements (reports).....	10
5.5	Transferts (transactions)	10
5.6	USSD : Pays / Opérateurs / Actions (ussd).....	10
5.7	Partenaires & affiliation (partners)	10
5.8	Alerte WhatsApp (whatsapp_alert)	11
5.9	Enquêtes, KPI, droits, notifications.....	11
6	Documentation Swagger / OpenAPI.....	12
7	Internationalisation (FR / EN).....	12
8	Sécurité renforcée (défense en profondeur).....	13
8.1	Authentification & identité.....	13
8.2	Autorisation (RBAC)	13
8.3	Protection des données	13
8.4	Sécurité applicative.....	13
8.5	Protection réseau & anti-attaques.....	14
8.6	Conformité	14
9	Communication avec le service IA	15
9.1	Mode synchrone — prédiction en temps réel.....	15
9.2	Mode asynchrone — apprentissage continu.....	15

9.3 Contrat d'échange & modèle partagé dans le dépôt Git.....	15
10 Observabilité, tests & déploiement.....	17
10.1 Journalisation & supervision.....	17
10.2 Tests	17
10.3 Déploiement.....	17
11 Livrables attendus.....	17

1 Présentation du document

Ce cahier des charges décrit de façon exhaustive le **backend hybride de KWISMO** : l'API centrale qui relie l'application mobile, le site web (landing, espaces Admin et Partenaire) et le service de modèle IA à une base de données. Le backend est la **seule porte d'accès à la base de données** : ni le front, ni le mobile, ni l'IA n'y accèdent directement. Ce document est la référence normative pour le développement (Python/FastAPI) et pour la génération de la documentation Swagger.

1.1 Rôle du backend dans l'architecture

- **Point central** : toutes les données transitent par le backend, qui applique l'authentification, les droits d'accès, la validation et la journalisation.
- **Fournisseur d'API** : expose une API REST documentée (Swagger/OpenAPI) consommée par le web et le mobile.
- **Passerelle IA** : transmet au service IA les données de vérification/signalement et récupère les scores de risque ; l'IA ne touche jamais la base directement (voir §9).
- **Multilingue** : sert les textes (notifications, alertes, messages d'erreur) en français et en anglais selon la préférence de l'utilisateur.

1.2 Objectifs de qualité

Exigence	Cible normative
Hybride BD	Compatible SQLite (défaut), PostgreSQL, MySQL et MongoDB via Prisma, sans réécriture du code métier.
Documentation	Swagger/OpenAPI généré automatiquement : routes, schémas, modèles et codes d'erreur.
Sécurité	Défense en profondeur : authentification forte, chiffrement, RBAC, protection réseau et anti-attaques.
Internationalisation	Tous les textes destinés à l'utilisateur disponibles en FR et EN.
Performance	Réponses rapides (opérations courantes < 200 ms hors traitement IA), pagination systématique.
Testabilité	Couche métier isolée, testable unitairement ; jeux de tests d'intégration sur l'API.

Convention de lecture

Les routes sont notées MÉTHODE /chemin (ex. POST /auth/login).

Les rôles sont : user (application mobile), partner (espace partenaire), admin (supervision globale).

Chaque module précise ses entités, ses routes et les règles d'accès par rôle.

2 Stack technologique

Domaine	Technologie	Justification
Langage	Python 3.11+	Demandé au cahier des charges ; écosystème riche, cohérent avec le service IA (même langage).
Framework API	FastAPI	Documentation OpenAPI/Swagger native, validation par types, hautes performances (ASGI), asynchrone.
Serveur ASGI	Uvicorn (+ Gunicorn)	Serveur de production performant ; Gunicorn gère plusieurs workers.
ORM	Prisma Client Python	Backend hybride : un seul schéma, plusieurs bases (SQLite/PostgreSQL/MySQL/MongoDB). Migrations intégrées.
Validation	Pydantic v2	Schémas d'entrée/sortie typés, cœur de la génération Swagger et de la sécurité des données.
Auth	JWT (python-jose) + Argon2	Jetons d'accès/rafraîchissement ; hachage de mots de passe robuste (argon2-cffi).
Documentation	Swagger UI + ReDoc (FastAPI)	Générés automatiquement à partir du code ; toujours à jour.
i18n	Fichiers de traduction (JSON) + middleware	Sélection FR/EN par en-tête de langue ; textes centralisés.
Tâches asynchrones	File d'attente légère (RQ/Celery selon besoin)	Traitement différé (envois, transmission des signalements à l'IA).
Cache	Redis (optionnel)	Cache de réponses, limitation de débit, files ; activable en production.
Tests	pytest + httpx	Tests unitaires et d'intégration de l'API.
Qualité	Ruff + Black + mypy	Lint, formatage et vérification de types.
Conteneurisation	Docker + docker-compose	Environnements reproductibles ; parité dev/prod ; déploiement simplifié.

Sur la compatibilité des bases de données

Prisma prend en charge SQLite, PostgreSQL, MySQL et MongoDB. Le développement démarre sur SQLite (zéro configuration) ; la production peut basculer vers PostgreSQL en changeant une seule variable d'environnement, sans modifier le code métier.

Cassandra n'est pas prise en charge par Prisma : elle est volontairement exclue du périmètre pour ne pas promettre une compatibilité non tenable.

3 Architecture & structure des dossiers

3.1 Choix : architecture en couches (Layered / Clean léger)

L'architecture retenue est une **architecture en couches modulaire par domaine**, largement recommandée par la communauté FastAPI (structure « feature-based » proche du dépôt de référence de N. Netfoundry / patterns officiels FastAPI) et par les principes de Clean Architecture, adaptée sans sur-ingénierie.

Pourquoi ce choix (justificatifs pour l'équipe)

- **Séparation des responsabilités** : les routes (présentation), la logique métier (services) et l'accès aux données (repositories/Prisma) sont séparés. On teste la logique sans lancer de serveur.
- **Modularité par domaine** : chaque module (auth, users, numbers...) regroupe ses routes, schémas et services. Ajouter un domaine ne perturbe pas les autres.
- **Sécurité centralisée** : l'authentification, les dépendances de rôle et la validation sont mutualisées dans une couche « core », appliquée uniformément.
- **Portabilité BD** : l'accès aux données passant par Prisma et une fine couche repository, changer de base n'impacte pas la logique métier.
- **Alternative écartée** : tout mettre dans les fichiers de routes (style script) rend le code instable et non maintenable dès que les règles se complexifient (rôles, affiliation partenaire...).

3.2 Arborescence normative

```

kwismo-backend/
├── app/
│   ├── main.py                # Point d'entrée FastAPI (montage routes, middlewares)
│   ├── core/                  # Cœur transverse
│   │   ├── config.py          # Variables d'environnement (Pydantic Settings)
│   │   ├── security.py        # JWT, hachage Argon2, dépendances d'auth
│   │   ├── permissions.py     # RBAC : dépendances require_role(admin/partner)
│   │   ├── rate_limit.py      # Limitation de débit
│   │   ├── middleware.py      # Journalisation, i18n, en-têtes de sécurité
│   │   ├── exceptions.py      # Erreurs métier + gestion centralisée
│   │   └── i18n.py             # Chargement des traductions, résolution de langue
│   ├── db/
│   │   ├── prisma_client.py   # Instance Prisma partagée (connexion)
│   │   └── repositories/      # Accès données par entité (fine abstraction)
│   └── modules/                # ★ Un domaine = un dossier
│       ├── auth/
│       │   ├── router.py       # Routes /auth/*
│       │   ├── schemas.py      # Pydantic (LoginIn, TokenOut...)
│       │   ├── service.py      # Logique (login, OTP, refresh, reset)
│       │   └── dependencies.py
│       ├── users/
│       ├── partners/
│       ├── numbers/
│       ├── reports/
│       ├── transactions/
│       └── ussd/

```

```

├── whatsapp_alert/      # Incident + diffusion d'alerte
├── surveys/             # Formulaires d'enquête
├── kpi/                 # Indicateurs (global & partenaire)
├── access_control/      # Rôles & droits
├── notifications/      # Notifications (FR/EN)
├── ai_gateway/          # Passerelle vers le service IA
├── locales/             # Traductions serveur
│   ├── fr.json         # messages, erreurs, notifications
│   └── en.json
├── utils/               # Helpers (phone, dates, pagination)
├── prisma/
│   ├── schema.prisma   # ★ Schéma unique (modèles = tables)
│   └── migrations/     # Historique des migrations
├── tests/               # pytest (unitaires + intégration)
├── scripts/             # Seed, tâches d'administration
├── .env / .env.example  # DATABASE_URL, JWT_SECRET, AI_SERVICE_URL...
├── Dockerfile
├── docker-compose.yml   # backend + base + (redis) + service IA
├── pyproject.toml       # Dépendances & outils (ruff, black, mypy)
└── README.md

```

3.3 Rôle des dossiers clés — sécurité, données, i18n

Élément	Responsabilité	Point de vigilance
core/security.py	JWT, hachage, dépendances d'authentification.	Source unique de la logique d'auth ; jamais dupliquée dans les modules.
core/permissions.py	Contrôle d'accès par rôle (RBAC).	Toute route protégée déclare explicitement le rôle requis.
db/repositories	Accès aux données via Prisma.	Isole le métier de la base ; facilite le changement de BD et les tests.
modules/ai_gateway	Communication avec le service IA.	Seul point qui parle à l'IA ; applique délais et repli si l'IA est indisponible.
locales	Textes FR/EN.	Aucun message utilisateur en dur dans le code ; tout passe par une clé de traduction.
prisma/schema.prisma	Modèle de données unique.	Vérité des tables ; toute évolution passe par une migration versionnée.

4 Modèle de données (Prisma)

Le schéma Prisma reprend le diagramme de classes du projet. Les entités ci-dessous sont les tables/collections principales. Chaque entité possède un identifiant, des horodatages de création/mise à jour, et des relations explicites.

Entité	Champs principaux	Relations
User	nom, prénom, numéro, pays, statut, rôle, dateInscription	Signalements, Transactions, Contacts, Réponses d'enquête, Rôle.
Contact	nom, numéro, statut (insigne)	Appartient à un User, correspond à un Numero.
Numero	valeur, scoreRisque, statut, dateDernièreVérif.	Opérateur, Signalements, Partenaire affilié.
Report (Signalement)	motif, date, statut	User (auteur), Numero (concerné).
Country (Pays)	nom, indicatif, estParDéfaut	Opérateurs.
Operator	nom, préfixes	Pays, Actions USSD, Numéros.
UssdAction	nomAction, format du code USSD	Opérateur.
Partner	nomEntreprise, type, dateAdhésion	Numéros affiliés, KPI, règles d'affiliation.
AffiliationRule	pays, préfixes/plages (69, 651-654...)	Partenaire, Pays.
Transaction	montant, date, statut, niveauRisque, codeUSSD	User, Numero, Pays, Opérateur, Action.
WhatsAppAlert	contenu, dateEnvoi, statut	User (émetteur), Contacts (destinataires).
SurveyResponse	question, réponse, date	User.
Kpi	indicateur, valeur, période, portée	Global ou Partenaire.
Role / AccessRight	nomRôle, permissions	Utilisateurs.
AuditLog	acteur, action, cible, date, IP	Traçabilité des actions sensibles.

Portabilité SQL / MongoDB

Le même schéma Prisma fonctionne en SQL (tables) et en MongoDB (collections). Les identifiants et relations sont déclarés de façon compatible.

Les champs de type liste (ex. préfixes d'une règle d'affiliation) sont modélisés de manière portable (table de liaison en SQL, tableau en MongoDB).

5 Modules fonctionnels & routes API

Chaque module expose des routes REST documentées automatiquement dans Swagger. Les accès sont contrôlés par rôle. La liste ci-dessous est normative (les noms exacts peuvent être ajustés à l'implémentation, mais la couverture doit être respectée).

5.1 Authentification (auth)

Route	Rôle	Description
POST /auth/register	public	Inscription (mobile) : pays + numéro.
POST /auth/otp/request	public	Envoi d'un OTP au numéro (durée de vie courte).
POST /auth/otp/verify	public	Vérification OTP → jetons d'accès/rafraîchissement.
POST /auth/login	public	Connexion email + mot de passe (admin/partenaire).
POST /auth/refresh	auth	Renouvellement du jeton d'accès.
POST /auth/password/forgot	public	Demande de réinitialisation par email.
POST /auth/password/reset	public	Réinitialisation via jeton reçu par email.
POST /auth/logout	auth	Révocation du jeton de rafraîchissement.

5.2 Utilisateurs (users)

Route	Rôle	Description
GET /users/me	user	Profil de l'utilisateur courant + KPI personnels.
PATCH /users/me	user	Mise à jour du profil.
GET /users	admin/partner	Liste paginée (partenaire : périmètre restreint).
GET /users/{id}	admin/partner	Détail d'un utilisateur.
PATCH /users/{id}/status	admin	Activer / suspendre.

5.3 Numéros (numbers)

Route	Rôle	Description
POST /numbers/verify	user	Vérifier un numéro (appelle l'IA) → score & statut.
POST /numbers/batch-verify	user	Vérifier une liste (contacts) en une fois.
GET /numbers	admin/partner	Liste paginée + filtres (statut, pays, opérateur).
GET /numbers/{id}	admin/partner	Détail + historique de vérification.
PATCH /numbers/{id}/status	admin/partner	Forcer un statut / réanalyse.

5.4 Signalements (reports)

Route	Rôle	Description
POST /reports	user	Créer un signalement (motif + numéro).
GET /reports	admin/partner	Liste des signalements (périmètre selon rôle).
PATCH /reports/{id}/validate	admin	Valider / clore un signalement.

5.5 Transferts (transactions)

Route	Rôle	Description
POST /transactions/prepare	user	Prépare le transfert : vérifie le numéro, renvoie le code USSD.
GET /transactions	user	Historique des transferts de l'utilisateur.
GET /transactions/{id}	user	Détail d'un transfert.

Le backend ne déplace pas d'argent

KWISMO génère et sécurise le code USSD ; la transaction financière réelle se déroule dans l'interface native de l'opérateur. Le backend ne stocke que des métadonnées (montant, statut, niveau de risque), jamais de code PIN.

5.6 USSD : Pays / Opérateurs / Actions (ussd)

Route	Rôle	Description
GET /countries	user	Liste des pays (drapeaux/indicatifs) — Cameroun par défaut.
GET /operators?country=	user	Opérateurs d'un pays.
GET /ussd-actions?operator=	user	Actions & formats USSD d'un opérateur.
POST /countries · /operators · /ussd-actions	admin	Création (CRUD complet réservé admin).
PATCH/DELETE ...	admin	Édition / suppression (admin).

5.7 Partenaires & affiliation (partners)

Route	Rôle	Description
GET /partners	admin	Liste des partenaires.
POST /partners	admin	Créer un partenaire.
GET /partners/{id}	admin	Fiche partenaire + KPI.
GET /partners/{id}/affiliation-rules	admin	Règles d'affiliation (pays + préfixes).
POST /partners/{id}/affiliation-rules	admin	Ajouter une règle (ex. 69, 651-654, 68).

Route	Rôle	Description
GET /partner/scope/*	partner	Vues restreintes au périmètre du partenaire (numéros, users, KPI).

Cloisonnement des données partenaire

Le filtrage par périmètre est appliqué CÔTÉ SERVEUR à chaque requête d'un partenaire : il ne voit que les numéros correspondant à ses règles d'affiliation, et les utilisateurs/KPI qui en découlent. L'admin peut consulter tout périmètre (mode supervision). Un partenaire n'accède jamais aux données d'un autre.

5.8 Alerte WhatsApp (whatsapp_alert)

Route	Rôle	Description
POST /whatsapp-alerts/incident	user	Déclarer un compte WhatsApp piraté.
POST /whatsapp-alerts/broadcast	user	Diffuser l'alerte aux contacts sélectionnés.

5.9 Enquêtes, KPI, droits, notifications

Route	Rôle	Description
GET /surveys/active · POST /surveys/{id}/answer	user	Récupérer et répondre à une enquête.
GET /kpi/global	admin	KPI globaux (dashboard admin).
GET /kpi/partner	partner	KPI du partenaire.
GET/POST /roles · /access-rights	admin	Gestion des rôles & permissions.
GET /notifications	user	Notifications de l'utilisateur (texte FR/EN).

6 Documentation Swagger / OpenAPI

- **Génération automatique** : FastAPI produit la spécification OpenAPI à partir du code (routes, schémas Pydantic, codes de réponse). Aucune documentation manuelle à maintenir.
- **Interfaces** : Swagger UI (interactif, « essayer la requête ») et ReDoc (lecture) exposés sur /docs et /redoc, protégés en production.
- **Contenu documenté** : chaque route porte un résumé, une description, les paramètres, le corps attendu, les réponses possibles et les erreurs ; les modèles décrivent chaque champ.
- **Regroupement** : les routes sont étiquetées par module (tags) pour une navigation claire dans Swagger.
- **Export** : la spécification openapi.json est exportable pour générer des clients (web/mobile) et pour l'intégration partenaire.

7 Internationalisation (FR / EN)

- **Portée** : tous les textes destinés à l'humain — notifications, alertes, messages d'erreur, libellés d'enquête, emails de réinitialisation — sont disponibles en français et en anglais.
- **Sélection de langue** : déterminée par l'en-tête de requête (Accept-Language) ou la préférence enregistrée de l'utilisateur ; défaut = français.
- **Organisation** : fichiers locales/fr.json et locales/en.json, avec des clés hiérarchiques (ex. errors.invalid_otp, notifications.suspicious_call). Aucun texte en dur dans le code.
- **Interpolation & pluriels** : variables insérées dans les messages (ex. « Numéro invalide pour {country} ») ; gestion des pluriels.

8 Sécurité renforcée (défense en profondeur)

La sécurité est traitée à plusieurs niveaux : identité, données, application et réseau. Aucun niveau ne suffit seul ; leur combinaison assure la robustesse.

8.1 Authentification & identité

- **JWT** : jeton d'accès de courte durée + jeton de rafraîchissement de plus longue durée, rotatif et révocable.
- **Mots de passe** : hachés avec Argon2 (résistant aux attaques par force brute et GPU) ; jamais stockés en clair.
- **OTP** : à durée de vie courte, à usage unique, invalidé après vérification.
- **Anti-force brute** : limitation des tentatives de connexion et d'OTP, verrouillage temporaire progressif.

8.2 Autorisation (RBAC)

- Contrôle d'accès basé sur les rôles (user / partner / admin) appliqué par des dépendances FastAPI sur chaque route protégée.
- Cloisonnement du périmètre partenaire appliqué systématiquement côté serveur (filtrage des données selon les règles d'affiliation).

8.3 Protection des données

- **Chiffrement en transit** : HTTPS/TLS obligatoire ; redirection du trafic non chiffré.
- **Chiffrement au repos** : chiffrement des champs sensibles (ex. données personnelles) et de la base selon l'hébergement.
- **Minimisation** : on ne collecte que le nécessaire ; les données transmises à l'IA sont réduites au strict besoin (voir §9).
- **Journal d'audit** : traçabilité des actions sensibles (qui, quoi, quand, IP) pour investigation.

8.4 Sécurité applicative

- Validation stricte des entrées (Pydantic) : rejet de toute donnée malformée avant traitement.
- Protection contre les injections (requêtes paramétrées via Prisma), XSS et CSRF (jetons, en-têtes adaptés).
- En-têtes de sécurité HTTP (HSTS, X-Content-Type-Options, X-Frame-Options, Content-Security-Policy).

- CORS strict : seules les origines du web et du mobile officiels sont autorisées.

8.5 Protection réseau & anti-attaques

- **Limitation de débit (rate limiting)** : plafonnement des requêtes par IP/utilisateur pour absorber les abus.
- **Reverse proxy / WAF** : passage par un proxy (ex. Nginx) et un pare-feu applicatif filtrant les requêtes malveillantes.
- **Atténuation DDoS** : protection au niveau de l'hébergeur/CDN contre les attaques par déni de service.
- **Isolation** : services conteneurisés, base non exposée publiquement, secrets hors du code (variables d'environnement / coffre).
- **Signature des échanges partenaires** : les webhooks/appels sortants vers les partenaires sont signés pour garantir l'authenticité.

8.6 Conformité

- Traitement des données personnelles conforme à la réglementation locale en vigueur ; le consentement des utilisateurs est recueilli par l'application mobile (hors périmètre de ce document).
- Politique de conservation et de suppression des données définie ; possibilité d'anonymisation.

9 Communication avec le service IA

Le service IA est un service Python séparé (voir cahier des charges dédié). Il **ne se connecte jamais directement à la base de données** : le backend est son unique interlocuteur. Deux modes de communication coexistent.

9.1 Mode synchrone — prédiction en temps réel

- Lors d'une vérification de numéro, le backend appelle le service IA (POST /predict) avec les caractéristiques nécessaires et reçoit un score de risque et un statut.
- **Robustesse** : délai maximal (timeout) défini ; en cas d'indisponibilité de l'IA, repli sur les règles expertes du backend (voir cahier IA) pour ne jamais bloquer l'utilisateur.

9.2 Mode asynchrone — apprentissage continu

- À chaque signalement/vérification, le backend transmet la donnée au service IA (file d'attente) qui met à jour son apprentissage et renvoie, le cas échéant, des statuts mis à jour que le backend enregistre.
- Aucune intervention manuelle d'administrateur : l'entraînement est déclenché automatiquement, conformément aux diagrammes du projet.

9.3 Contrat d'échange & modèle partagé dans le dépôt Git

Le backend et le service IA vivent dans **le même dépôt Git**, dans deux sous-dossiers distincts (kwismo-backend/ et kwismo-ai/). Pour que la communication soit sans friction, trois règles sont imposées.

1. **Contrat d'API partagé** : les formats d'échange (entrée /predict, sortie score/statut) sont définis par des schémas communs, documentés dans les deux Swagger, afin qu'aucun des deux services ne « devine » le format de l'autre.
2. **Artefact de modèle accessible** : quand l'entraînement produit un nouveau modèle, celui-ci est publié dans un emplacement convenu (dossier partagé monté en volume, ou registre d'artefacts). Le service IA le charge pour l'inférence ; le backend n'a pas besoin du fichier de modèle, il appelle seulement l'API IA.
3. **Orchestration commune** : un docker-compose à la racine du dépôt démarre ensemble le backend, la base et le service IA, avec les variables (AI_SERVICE_URL, chemins de modèle) déjà câblées. Un seul « docker compose up » lève tout l'environnement.

Résultat attendu

Dès que l'entraînement se termine, le nouveau modèle est immédiatement exploitable par le service IA, et donc par le backend, sans manipulation manuelle ni incompatibilité de format. Les deux dossiers restent indépendants (équipes séparées) mais parlent le même langage grâce au contrat d'API partagé et à l'orchestration commune.

10 Observabilité, tests & déploiement

10.1 Journalisation & supervision

- Journaux structurés (JSON) : requêtes, erreurs, événements de sécurité, appels IA.
- Points de contrôle de santé (/health) pour la supervision et l'orchestration.

10.2 Tests

- Tests unitaires de la logique métier (services) ; tests d'intégration des routes (pytest + httpx).
- Jeux de données de test (seed) reproductibles ; base SQLite en mémoire pour les tests rapides.

10.3 Déploiement

- Conteneurisation Docker ; base non exposée publiquement ; secrets en variables d'environnement.
- Migrations Prisma appliquées au déploiement ; bascule SQLite → PostgreSQL par simple changement de DATABASE_URL.
- Intégration continue : lint, types et tests exécutés à chaque poussée de code.

11 Livrables attendus

4. API FastAPI complète couvrant tous les modules du §5, protégée par la sécurité du §8.
5. Schéma Prisma + migrations, fonctionnant sur SQLite (dev) et prêt pour PostgreSQL (prod).
6. Documentation Swagger/OpenAPI générée et exportable (openapi.json).
7. Fichiers de traduction FR/EN complets.
8. Passerelle IA (synchrone + asynchrone) avec contrat d'API partagé et repli par règles.
9. Configuration Docker + docker-compose orchestrant backend + base + service IA.
10. Suite de tests (unitaires + intégration) et jeux de données de seed.